Boosting Methods — Hinglish Notes

Nov 8, 2026 · @Razeen

Yeh notes Atlas Skilltech University ke ML course ke liye hain. Boosting Methods — ek powerful ensemble technique jo industry mein widely use hoti hai.

1. Introduction to Boosting

Ensemble Learning ka matlab hai kai saare models ko combine karke ek powerful model banana. Jaise ek akela doctor kam experienced hota hai, lekin agar 100 doctors mil ke decision lein toh accuracy zyada hogi — yahi ensemble learning hai.

Bagging vs Boosting

Feature	Bagging	Boosting
Training Style	Parallel (sab ek saath)	Sequential (ek ke baad ek)
Error Focus	Random sampling	Pichle model ki galtiyon pe focus
Example	Random Forest	AdaBoost, GBM
Overfitting Risk	Kam	Thoda zyada (par generally better)
Speed	Fast	Slow (sequential)
Use Case	High variance problems	High bias problems

Weak Learner vs Strong Learner

Weak Learner: Ek aisa model jo random se thoda better ho. Example: Decision Stump (depth=1 ka decision tree) — sirf ek sawaal poochta hai.

Strong Learner: Kai weak learners ka combination jo high accuracy deta hai.

Key Idea: Boosting mein weak learners ek ke baad ek train hote hain. Har naya learner pichle learner ki galtiyon pe zyada focus karta hai.

Real Life Example

Socho ek student exam mein fail hota hai. Pehla teacher basics sikhata hai. Doosra teacher unhi topics pe zyada focus karta hai jahan student fail hua tha. Teesra teacher aur bhi difficult questions practice karata hai. Yahi boosting ka concept hai!

2. Boosting Methods Overview

Aaj kai saare boosting algorithms available hain. Yeh comparison table important hai:

Algorithm	Year	Key Feature	Best For
AdaBoost	1997	Sample weights update karta hai	Small datasets, clean data
Gradient Boosting (GBM)	1999	Residuals pe train karta hai	Medium datasets
XGBoost	2014	Regularization + parallel processing	Kaggle competitions
LightGBM	2017	Leaf-wise growth, faster	Large datasets
CatBoost	2017	Categorical features auto-handle	Mixed data types

Industry Mein Kaise Use Hote Hain?

- **AdaBoost:** Medical diagnosis, face detection (Viola-Jones algorithm)
- **GBM:** Credit risk scoring, insurance pricing
- **XGBoost:** Kaggle competitions, fintech fraud detection
- **LightGBM:** Real-time recommendation systems (Netflix, Amazon)
- **CatBoost:** E-commerce click prediction (Yandex)

3. AdaBoost Theory

AdaBoost = Adaptive Boosting — 1997 mein Freund aur Schapire ne develop kiya.

Algorithm Steps

1. Sab samples ko equal weight do: w_i = 1/N
2. Ek weak learner (Decision Stump) train karo
3. Error rate calculate karo:

$$\varepsilon_t = \frac{\sum_{i=1}^{N} w_i \cdot \mathbb{1}[y_i \neq h_t(x_i)]}{\sum_{i=1}^{N} w_i}$$

4. Learner ka weight (alpha) calculate karo:

$$\alpha_t = \frac{1}{2} \ln\left(\frac{1 - \varepsilon_t}{\varepsilon_t}\right)$$

5. Sample weights update karo — galat classified samples ka weight badhao:

$$w_i^{(t+1)} = w_i^{(t)} \cdot \exp(-\alpha_t \cdot y_i \cdot h_t(x_i))$$

6. Weights normalize karo (sum = 1)
7. Steps 2–6 repeat karo T rounds tak
8. Final prediction:

$$H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t \cdot h_t(x)\right)$$

Key Formulas Samjhao

Formula	Matlab
ε_t near 0	Perfect weak learner, α bahut bada
ε_t = 0.5	Random guessing, α = 0 (koi contribution nahi)
ε_t near 1	Jaan-boojh ke galat, α negative

Decision Stump: max_depth=1 ka tree — sirf ek feature pe ek threshold check karta hai. AdaBoost ka default base estimator yahi hai.

4. AdaBoost Implementation (sklearn)

Basic Code

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

# AdaBoost with Decision Stump
ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),  # Weak learner
    n_estimators=100,     # Kitne weak learners
    learning_rate=1.0,    # Har learner ka contribution
    algorithm='SAMME',    # Discrete AdaBoost (sklearn 1.4+)
    random_state=42
)
ada.fit(X_train, y_train)
```

Key Hyperparameters

Parameter	Default	Matlab
n_estimators	50	Kitne weak learners train karne hain
learning_rate	1.0	Har learner ka contribution scale factor
estimator	DecisionTreeClassifier(max_depth=1)	Base weak learner
algorithm	'SAMME'	Discrete (SAMME) ya continuous (deprecated)

Evaluation

```
# Old Way
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
cm = confusion_matrix(y_test, ada.predict(X_test))
plt.imshow(cm, cmap='Blues')

# New Way (1 line!)
from sklearn.metrics import ConfusionMatrixDisplay
ConfusionMatrixDisplay.from_estimator(ada, X_test, y_test, cmap='Blues')

# Learning Curve
train_scores = list(ada.staged_score(X_train, y_train))
test_scores = list(ada.staged_score(X_test, y_test))
```

Common Mistakes

- **algorithm='SAMME.R'** deprecated hai sklearn 1.4+ mein — `SAMME` use karo
- **n_estimators bahut bada** = overfitting ho sakta hai
- **learning_rate bahut bada** = unstable training

5. Gradient Boosting Theory

Gradient Boosting = har tree ko pichle model ki **residuals (galtiyon)** pe train karna.

Core Idea (Regression Example)

Maan lo hum ghar ki price predict kar rahe hain:

1. Pehle simple prediction: mean = 300K
2. Residuals (errors): [+50K, -20K, +30K, ...]
3. Naya tree in residuals pe train hota hai
4. Update: F_1(x) = 300K + 0.1 * tree_1(x)
5. Step 2-4 repeat karo

Mathematical Formulation

Initial prediction (mean):

$$F_0(x) = \arg\min_{\gamma} \sum_{i=1}^{N} L(y_i, \gamma)$$

Har round mein residuals (pseudo-residuals):

$$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}}$$

Simplified (MSE loss ke liye): $r_i = y_i - F(x_i)$

Model update har round mein:

$$F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)$$

Jahan ν = learning rate (shrinkage), $h_m(x)$ = naya tree

Key Concepts

Concept	Matlab
Residual	Actual - Predicted (galti)
Learning Rate (ν)	Har tree ka contribution kitna ho (0 se 1)
Subsample	Har tree ke liye kitna data use karna (Stochastic GB)
max_depth	Har tree kitna deep ho (usually 3-5)
n_estimators	Total kitne trees (usually 100-500)

Tradeoff: Low learning rate + zyada trees = better (but slower). Practically: lr=0.1, n=100 se start karo.

6. Gradient Boosting Implementation (sklearn)

Basic Code

```
from sklearn.ensemble import GradientBoostingClassifier

gbm = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    subsample=0.8,       # Stochastic GB — 80% data per tree
    max_features='sqrt', # Random feature selection
    random_state=42
)
gbm.fit(X_train, y_train)
```

AdaBoost vs GBM Comparison

Feature	AdaBoost	Gradient Boosting
Focus	Sample weights	Residuals
Loss Function	Exponential loss	Any differentiable loss
Overfitting	Resistant	More prone
Speed	Faster	Slower
Flexibility	Less	More
Noise Sensitivity	High	Moderate

Feature Importance

```
import pandas as pd
import matplotlib.pyplot as plt

feat_imp = pd.Series(gbm.feature_importances_, index=X.columns)
feat_imp.nlargest(10).sort_values().plot(kind='barh')
plt.title('Feature Importances')
plt.show()
```

Staged Prediction (Learning Curve)

```
# Har tree ke baad test accuracy track karo
train_scores = list(gbm.staged_score(X_train, y_train))
test_scores = list(gbm.staged_score(X_test, y_test))

plt.plot(train_scores, label='Train')
plt.plot(test_scores, label='Test')
plt.xlabel('n_estimators')
plt.ylabel('Accuracy')
plt.legend()
```

7. Hyperparameter Tuning — GridSearchCV

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [50, 100, 200],
    'learning_rate': [0.05, 0.1, 0.2],
    'max_depth': [2, 3, 4]
}

grid = GridSearchCV(
    GradientBoostingClassifier(subsample=0.8, random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1   # Sab CPU cores use karo
)
grid.fit(X_train, y_train)

print(grid.best_params_)
print(grid.best_score_)
```

Tips for Tuning

Parameter	Recommendation
n_estimators	Pehle 100 se start karo, then increase
learning_rate	0.1 good default, 0.01-0.3 range explore karo

Parameter	Recommendation
max_depth	3-5 (zyada = overfitting)
subsample	0.7-0.9 (Stochastic GB ke liye)
min_samples_split	2-20 (regularization)

8. Real World Applications + Summary

Real World Use Cases

Industry	Use Case	Algorithm
Healthcare	Cancer detection, disease risk	AdaBoost, GBM
Finance	Credit scoring, fraud detection	XGBoost, GBM
E-commerce	Click prediction, recommendations	LightGBM
Insurance	Claim prediction, pricing	GBM, XGBoost
HR	Employee attrition prediction	AdaBoost
Search Engine	Ranking results	GBM variant

Complete Concepts Summary

Concept	Matlab	Key Point
Weak Learner	Thoda better than random	Decision Stump (depth=1)
Ensemble	Kai models combine karo	Final = weighted sum
AdaBoost Alpha	Learner weight	0.5 x ln((1-e)/e)
GBM Residual	Galti	r_i = y_i - F(x_i)
GBM Update	Model improve karo	F_m = F_{m-1} + lr x h_m
Learning Rate	Contribution control	Usually 0.01-0.3
Staged Score	Round-by-round accuracy	model.staged_score()

Concept	Matlab	Key Point
Feature Importance	Kaunsa feature useful	model.feature_importances_

Aage Kya Seekhein?

1. **XGBoost** - pip install xgboost, same API as sklearn
2. **LightGBM** - Large datasets ke liye, Microsoft ka
3. **CatBoost** - Categorical features auto handle karta hai
4. **SHAP Values** - Model decisions explain karna
5. **Stacking** - Multiple diverse models combine karna